

OptoFlood: Controllable Flooding for NDN Producer Mobility

Yuting Wan
University of Glasgow
yuting.wan@glasgow.ac.uk

Ryo Yanagida
University of Glasgow
ryo@htonl.net

Paul Harvey
University of Glasgow
paul.harvey@glasgow.ac.uk

Jeremy Singer
University of Glasgow
jeremy.singer@glasgow.ac.uk

Colin Perkins
University of Glasgow
csp@csperkins.org

Leon Wong
Rakuten Mobile, Inc.
leon.wong@rakuten.com

Abstract—Producer mobility in systems that use Named Data Networking (NDN) protocols can lead to increased latency and packet loss due to slow routing convergence and stale forwarding state. This affects applications, such as live video streaming and video conferencing, where even a brief interruption in packet delivery can degrade user experience. To address this, we propose *OptoFlood*, a controllable flooding mechanism that supports both Data and Interest delivery immediately following producer movement events, to rapidly establish temporary bidirectional communication paths. In *OptoFlood*, mobility-marked packets are disseminated with bounded scope, deduplication, and rate limiting, while intermediate nodes create short-lived forwarding hints via a temporary FIB (TFIB). When forwarding fails due to both FIB and TFIB misses, the Forwarder triggers a hop-limited flooding procedure to discover viable paths. *OptoFlood* also accelerates global routing convergence through Fast-LSA. Our evaluation demonstrates that *OptoFlood* effectively reduces latency and packet loss caused by producer mobility while maintaining reasonable network overhead.

Index Terms—Named Data Networking, Producer Mobility, Controlled Flooding, TFIB, NLSR

I. INTRODUCTION

The use of Named Data Networking (NDN) protocols optimises the network for content delivery [1], [2]. Rather than relying on an indirect name-to-location mapping via the DNS, as is done in the Internet today, NDN protocols make name-based routing, content delivery, and pervasive caching core services of the network.

However, a limitation of current NDN protocols is that they do not effectively support producer mobility. Each time a content producer moves, the network must quickly discover its new location and ensure continuous data delivery without introducing delays or packet loss. For applications such as live video streaming and video conferencing, even short interruptions can degrade user experience and reliability. In practice, producer movement invalidates previously valid forwarding paths, while routing updates may take seconds to converge. During this period, Interest packets can be misrouted, timed out, or discarded due to a stale forwarding state. Previous works have improved the behaviour of NDN in these scenarios, but still lag behind what is needed [3]–[6].

In this paper, we introduce *OptoFlood*, a novel controlled flooding scheme that can establish temporary bidirectional communication paths to accelerate network convergence after producer mobility events. *OptoFlood* proactively manages controlled local flooding of both *Data* and *Interest* packets, enabling rapid recovery from disruptions caused by producer movements.

Specifically, *OptoFlood* includes two complementary components.

First, after a producer movement event, the producer performs *Data packet flooding* for mobility-marked Data corresponding to pending Interests. This flooding is explicitly bounded (e.g., hop-limited) and rate controlled, and is intended to quickly “push” Data across nearby nodes so that Interests that would otherwise stall can be satisfied without waiting for global routing convergence. Second, when forwarding fails because both the FIB and a temporary forwarding table (TFIB) provide no usable next hop, forwarders trigger a hop-limited *Interest packet flooding* procedure to promptly rediscover viable paths toward the producer’s new attachment point. Intermediate nodes opportunistically create short-lived TFIB hints based on newly observed forwarding opportunities, which helps suppress repeated flooding and restore unicast forwarding quickly.

OptoFlood also integrates with Named-data Link State Routing (NLSR) by generating short-lived routing updates based on newly established temporary paths (Fast-LSA), which accelerates global routing convergence. Collectively, these mechanisms reduce latency and packet loss while maintaining reasonable overhead in network resource usage.

Compared to previous NDN mobility protocols, such as routing-based updates [7], Interest-based notification solutions [8] or anchor-based tracing mechanisms [9], *OptoFlood* provides a more integrated and lightweight solution that leverages flooding as its primary mechanism for rapid path recovery and convergence. Unlike existing flooding solutions **CSP: add citations**, our approach uses explicit controls, including hop limits, deduplication within a short time window, and producer-side flood rate limits, to restrict the scope and duration of flooding, minimising unnecessary overhead. Addition-

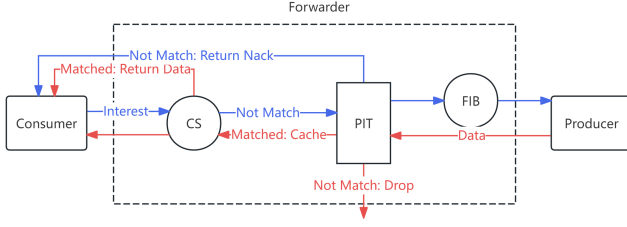


Fig. 1. NDN Packets Processing Flow

ally, by leveraging temporary bidirectional paths established via controlled flooding and propagating them through short-lived routing updates, OptoFlood accelerates global routing convergence without relying on centralised anchor nodes.

We structure the remainder of this paper as follows. Section II reviews the background of producer mobility problems in NDN, highlighting the limitations of traditional routing updates. Section III presents the detailed design of OptoFlood, including the mechanisms for controlled flooding of Data and Interest packets, and the integration with NLSR to accelerate global convergence. Section IV describes our experimental setup and presents results that demonstrate the effectiveness of our approach. We discuss related work in Section V. Finally, Section VI concludes the paper and briefly discusses potential directions for future enhancements of the OptoFlood approach.

II. PRODUCER MOBILITY IN NDN

NDN shifts from a host-centric model to a data-centric model, where content is identified by names rather than addresses [10], [11]. In NDN, communication relies on two core packet types: Interest Packets, sent by consumers to request data by specifying the content name, and Data Packets, returned by producers containing the requested content and related metadata. Both types of packets follow a concise Type-Length-Value (TLV) encoding, allowing efficient processing and extendability [1], [2].

NDN packet forwarders maintain a Content Store that caches Data packets; a Pending Interest Table (PIT) that records forwarded Interest packets awaiting the corresponding Data packets, ensuring accurate reverse path forwarding; and a Forwarding Information Base (FIB) that guides Interest packet forwarding [2], [12]. When an Interest arrives at a forwarder, it checks the Content Store and returns the cached Data, if it is available. Otherwise, the PIT is checked to aggregate Interests, and if the Interest is new, it is forwarded based on the FIB. Figure 1 illustrates packet forwarding.

The Named-Data Link State Routing protocol (NLSR) disseminates routing information via Link State Advertisements (LSAs) that carry link status and available data prefix information that are used to populate the FIB. NLSR adapts to network changes, such as node failures, new node attachments, or topology changes through periodic updates [13], [14].

Producer mobility in NDN poses significant challenges, especially in dynamic environments. When a producer moves,

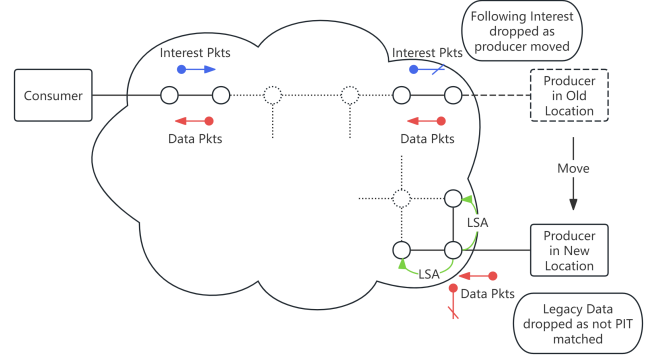


Fig. 2. NDN Producer Mobility Problem **RY: should this not be side-by-side with the solution Fig.5?**

it must re-register its data prefix so that NLSR can propagate the new route. Convergence is not immediate and is slow relative to mobility. During the convergence period, Interests may continue to follow stale forwarding state towards the producer's previous attachment point, while the producer begins responding at the new location. This mismatch can leave pre-move Interests pending for an extended time, and can also prevent newly generated Data from being returned successfully along the reverse path. Figure 2 provides an illustrative example of this handover behaviour.

This results in the following.

- **Increased Latency:** When a producer changes its attachment point, its associated data prefixes must be re-advertised, initiating routing updates starting from the directly connected forwarder. These updates must propagate throughout the network before Interest Packets can be delivered to the producer at its new location. During this convergence period, consumers experience significant communication interruptions, often lasting several seconds or longer [3]. In scenarios requiring low latency, such as live video streaming or real-time conferencing, even brief interruptions degrade the user experience and severely limit the practical utility of NDN [3]. Figure 2 illustrates how the service resumes only after the routing information has propagated sufficiently for Interests to reach the new attachment point.
- **Packet Loss:** During producer relocation and the subsequent routing convergence process, Interest packets following stale forwarding state may be unable to reach the producer's new location, causing these packets to expire and be discarded. Additionally, Data packets generated in response to Interests that were issued shortly before relocation may fail to return successfully if the reverse path state has been invalidated. In particular, Data may encounter forwarders that do not have a matching PIT entry (e.g., because the Interest was forwarded along a different path, or because the PIT state has timed out), leading to dropped Data and further degradation of communication reliability.

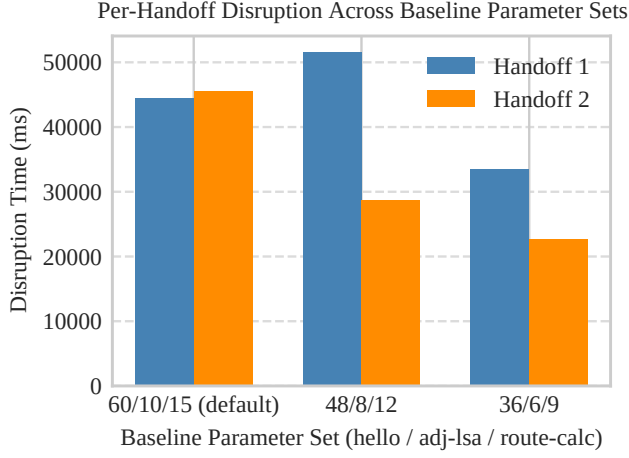


Fig. 3. Service disruption under baseline NDN forwarding with different NLSR parameter settings. Each label reports the hello, adjacency-LSA build, and routing calculation intervals in seconds.

- **Increased Network Resource Usage:** Inefficiencies introduced by latency and packet loss also result in increased retransmissions of packets of Interest. These retransmissions consume additional bandwidth and processing resources across the network, potentially impacting overall network performance and affecting other communications.

To test whether NLSR parameter tuning alone can address this limitation, we measured baseline forwarding under three NLSR configurations that progressively reduce the hello interval, adjacency-LSA build interval, and routing calculation interval. Figure 3 shows that the lowest-interval configuration tested (36/6/9 s) reduces the observed interruption relative to the default configuration in some hand-offs, but still leaves service disruption on the order of tens of seconds. Figure 4 shows the corresponding forwarding and control-plane cost. Across the tested configurations, the full-run forwarding-cost ratio remains close to 5.1–5.2, while NLSR control traffic increases as intervals are shortened.

These problems highlight the limitations of traditional routing-based approaches, such as NLSR, in handling frequent producer mobility. Addressing these issues effectively requires specialised mechanisms that can rapidly establish temporary communication paths without waiting for global routing convergence.

III. CONTROLLABLE FLOODING FOR NDN MOBILITY

To address the challenges of producer mobility in NDN, particularly in latency-sensitive applications such as video conferencing or live streaming, we propose *OptoFlood*, a novel controllable flooding mechanism. OptoFlood rapidly establishes temporary bidirectional communication paths following producer movements and leverages these paths to accelerate routing convergence.

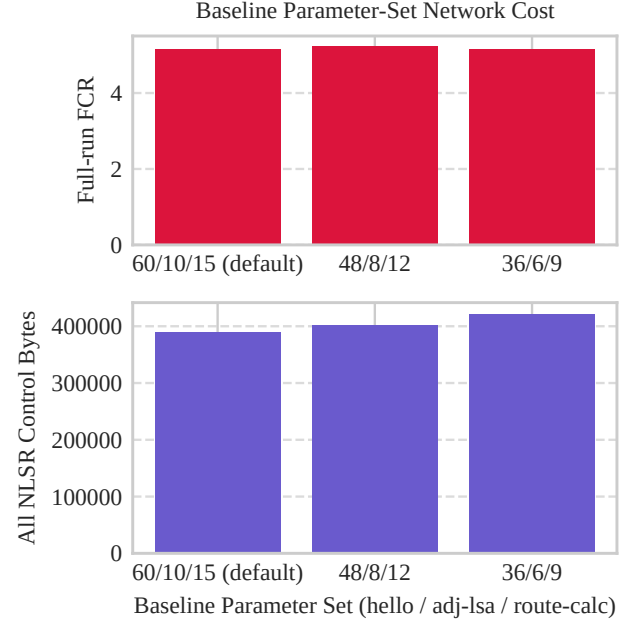


Fig. 4. Forwarding and NLSR control cost under the same baseline parameter settings.

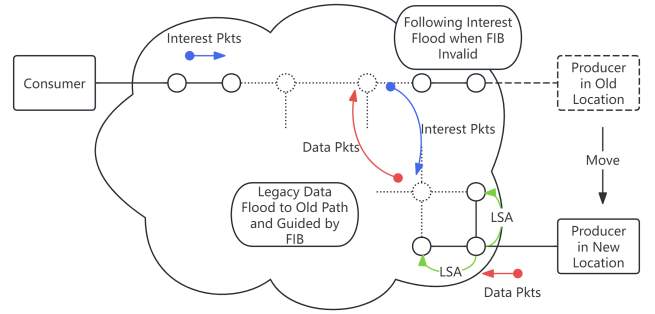


Fig. 5. NDN Producer Mobility Problem Solution

A. Producer Mobility Detection and Data Packet Flooding

In our implementation, producer mobility is detected at the application layer (e.g., via interface-up events), and the producer marks subsequent Data packets to indicate a mobility event. Flooding is then executed by forwarders when they process such mobility-marked Data, rather than by the producer directly; mobility triggers based on attachment/interface events are commonly used in mobility systems [15], [16].

Each mobility-marked Data packet carries the following markings:

- **LP-MobilityFlag:** a hop-by-hop flag in the LP header used to identify mobility-related Data at forwarders. If a forwarder finds a matching PIT entry, it clears the flag and forwards the Data normally so that the return path proceeds through standard PIT-based forwarding.
- **LP-OptoHopLimit:** a hop-by-hop scope control decre-

mented at each hop (default 3) to confine Data flooding to a local region. This field is experimental in our deployment and is distinct from the native Interest HopLimit.

- **Flood-ID**: an immutable identifier used for duplicate suppression in floods. In our implementation, FloodId is encoded as a Data MetaInfo extension TLV so that it is part of the signed portion of the Data and cannot be forged or altered by intermediate nodes.
- **NewFaceSeq**: an immutable sequence number used by forwarders to create or update a Temporary FIB (TFIB) entry upon receiving mobility-marked Data. NewFaceSeq is also carried in the signed portion of the Data (MetaInfo extension TLV) to ensure integrity.

At the forwarding plane, a forwarder treats a Data packet as OptoFlood-enabled when it detects FloodId/NewFaceSeq in the Data MetaInfo. The forwarder enforces duplicate suppression based on Flood-ID and applies a per-producer rate limit to bound overhead.

For dissemination, flooded Data packets *first* follow available FIB next hops. If no valid FIB next hop exists, forwarders fall back to *scoped expansion* over data-plane adjacent faces under LP-OptoHopLimit, excluding LOCAL faces and suppressing the incoming face (and any already satisfied faces) to avoid immediate loops. The aim is to reach a node that still holds a valid Pending Interest Table (PIT) entry along (or near) the original path so that the Data can be returned to the consumer without waiting for global routing updates.

B. Forwarder-Triggered Interest Flooding

Interest flooding is also performed by the forwarder and is triggered only as a last resort. In our implementation, the flooding procedure is invoked on a Content Store miss when both the FIB and the TFIB provide no usable next hop for the Interest prefix.

The forwarder sets and enforces the native *HopLimit* (default 3) before flooding (if not already present), decrementing it at each hop. Interest flooding is sent only to non-LOCAL faces.

To prevent repeated storms, each forwarder maintains a short-lived per node cache keyed by (Name, Nonce) so that the same Interest is flooded at most once per node within a small time window (about one second).

We also retain an explicit triggering path: if an Interest already carries a HopLimit, the forwarder may treat it as an explicit request to flood once even when a FIB entry exists, although the default consumer in our evaluation does not rely on this option.

C. Temporary Bidirectional Path Establishment

A key goal of OptoFlood is to establish a temporary *bidirectional* communication capability quickly after handover. In the downstream direction (consumer → producer), Interest flooding can locate a viable path toward the producer's new attachment point when unicast forwarding fails. In the upstream direction (producer → consumer), mobility-marked

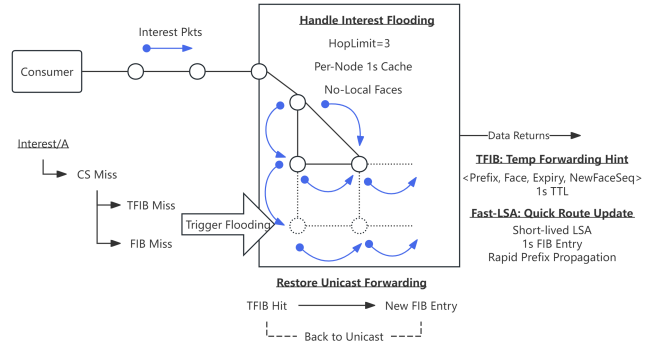


Fig. 6. Interest Flooding

Data flooding increases the likelihood that Data corresponding to Interests issued shortly before handover can reach a forwarder that still maintains the appropriate PIT state.

In addition, forwarders opportunistically create short-lived forwarding hints via a *Temporary FIB (TFIB)*. Upon receiving mobility-marked Data, a forwarder creates or updates a TFIB entry (separate from the standard FIB) mapping the relevant prefix to the incoming face, annotated with NewFaceSeq/FloodId, and with a short TTL (about one second). Subsequent Interests can then switch to unicast via TFIB immediately, reducing the need for repeated Interest flooding while global routing converges [9], [17].

D. Accelerated Network Convergence

OptoFlood integrates temporary flooding-based paths with faster global convergence via NLSR. When a forwarder successfully establishes or updates a TFIB entry, it triggers an NLSR management command at `/localhost/nlsr/fast-lsa/trigger`. This command installs a short-lived routing entry (Fast-LSA style) with an expiration period of approximately one second, and employs throttling (500 ms per prefix) and de-duplication (by Prefix+NewFaceSeq) to limit control-plane overhead [13].

Regular periodic LSAs later supersede short-lived updates, and TFIB entries expire naturally.

E. Security and Flooding Control

To mitigate risks associated with flooding, OptoFlood uses complementary control mechanisms.

Digital signatures for immutable fields. NDN Data packets are signed; immutable fields such as Names, Content, and the MetaInfo extensions (e.g., FloodId/NewFaceSeq) are authenticated end-to-end. Dynamic hop-by-hop controls (*LP-MobilityFlag*, *LP-OptoHopLimit*) live in the LP header outside the signature so that intermediate nodes can legitimately decrement or clear them without breaking integrity checks on the Data itself.

Scoped hop limits. A small default hop limit (3) confines both Data and Interest flooding to a local region and prevents excessive scope. Packets exceeding the threshold are dropped.

Rate limiting and duplicate suppression. Forwarders enforce a per-producer flood rate limit (e.g. 100 packets/s for flooded traffic) and maintain a FloodId-based cache for duplicate suppression. Together these controls bound overhead and reduce the attack surface for denial-of-service attempts, while still allowing enough exploration to rescue pending Interests.

Together, these controls bound overhead and reduce the attack surface for denial-of-service attempts, while still allowing enough exploration to rescue pending Interests [18].

IV. EVALUATION

We evaluate OptoFlood in a real-time streaming scenario with local producer mobility. The objective is to determine whether temporary flooding and Fast-LSA can preserve service continuity during hand-off while keeping the additional forwarding and control traffic bounded.

A. Methodology

We consider producer micro-mobility within a single access domain, where an ongoing real-time stream, generated by the producer in response to received Interest packets, remains active while the producer changes its attachment point. This setting stresses the interval between the physical move and routing convergence, which is precisely where NDN producer mobility causes service interruption.

1) *Producer Application:* We implement a typical NDN video producer with our mobility extensions. Generated video Data packets have a freshness period of 10 seconds, permitting caching but preventing stale data from clogging the network, and carry a payload of synthetic data matching the size and timing of video content. The packets are digitally signed to ensure integrity and security. Content prefixes are advertised via NLSR in the usual manner, with the extensions discussed in Section III.

CSP: In the following paragraph, “marks the Data packets associated with the mobility event” is imprecise. Should it be something like “identifies Data packets that will be sent after the change in attachment point but in response to an Interest received before the change”? Also need to cross-reference the earlier section with details of the mechanism. Upon detecting changes in its network attachment point, the producer marks the Data packets associated with the mobility event to enable OptoFlood to control the flooding and acceleration of convergence. In our deployment, immutable identifiers used for flooding (e.g., Flood-ID and a sequence number for forwarding hints) are carried in signed Data MetaInfo extension TLVs, while hop-by-hop controls (e.g., mobility indication and scoped hop limits) are carried in the LP header and may be updated by intermediate forwarders.

2) *Consumer Application:* The consumer issues Interests every 33 ms (30 fps) with freshness and a 6 s lifetime, validates returned Data using a trust schema, and logs basic metrics.

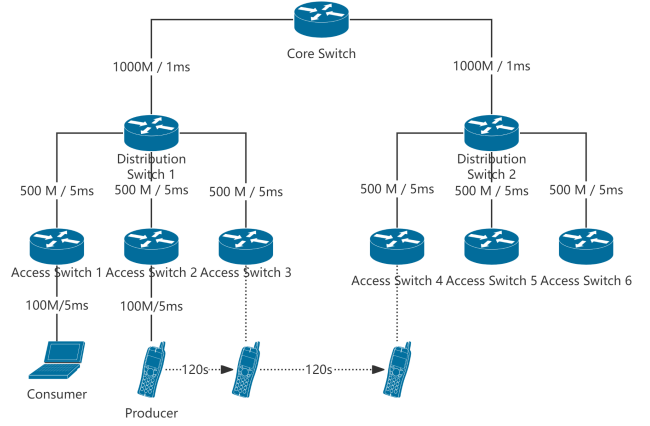


Fig. 7. Experiment topology used in the mobility evaluation. **RY: change the style of topology diagram to be consistent with Figure 6**

3) *Network Topology:* The evaluation topology is a three-layer access network with one core node, two aggregation nodes, and six access nodes. The consumer is attached to one access node, while the producer is initially attached to a neighbouring access node and then moves to two alternative access nodes during the run. This setup creates local hand-offs that remain within a few hops but still require the forwarding plane and routing plane to react.

The links are configured as follows:

- **Core to aggregation:** 1 Gbps, 1 ms.
- **Aggregation to access:** 500 Mbps, 5 ms.
- **Consumer/producer to access:** 100 Mbps, 5 ms.

4) *Experiment Procedure and Measurement:* The experiments are run in Mini-NDN 0.7.0 [19] with identical topology, applications, and hand-off schedule in two configurations: baseline NDN forwarding and OptoFlood-enabled forwarding. In each run, all nodes start NFD and NLSR and are given time to converge before the applications are launched. The producer then performs two scheduled hand-offs, separated by long enough intervals **CSP: How long? Be specific.** to let the transient behaviour around each event be observed clearly.

We record packet traces (ndndump) in three places: the consumer, the access switch adjacent to the old PoA, and the access switch adjacent to the new PoA for each hand-off. We additionally retain packet traces on all relay and endpoint nodes for analysis. **CSP: What is a relay node? This term has not been defined.** The simulation runs as containers on a single AMD EPYC server with 64 cores, 128 threads, 512GB memory, running Debian 12 “bookworm”. Where cross-node timing is required, we compute relative times from the consumer’s point of view.

We report four complementary views of performance:

- **Throughput continuity:** delivered consumer throughput over time.
- **Service disruption time:** for each hand-off, the time between the last Data packet observed before the move

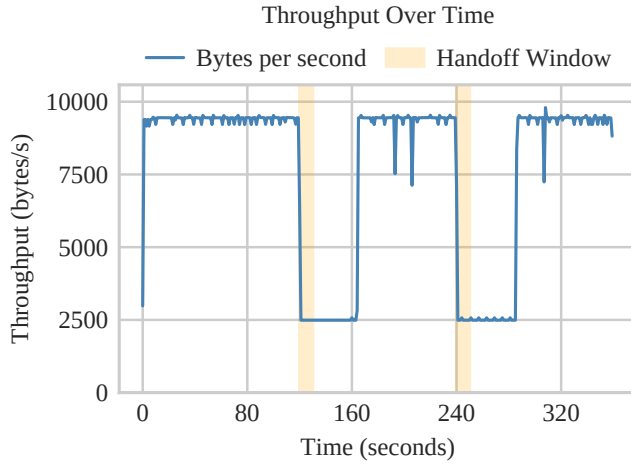


Fig. 8. Throughput continuity under baseline NDN forwarding with the default NLSR parameter setting.

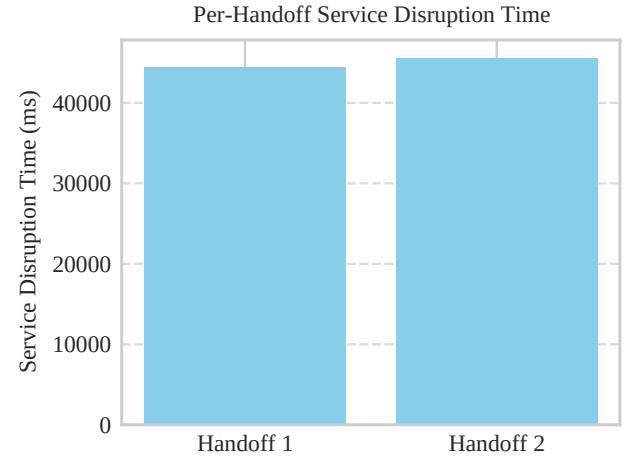


Fig. 9. Per-handoff service disruption under baseline NDN forwarding with the default NLSR parameter setting.

and the first Data packet observed after service resumes.

CSP: Observed where?

- **Unmet-interest ratio:** an Interest is counted as unmet if the first request for a content name is not satisfied by Data within the application deadline; we evaluate this in the 10s interval immediately after each hand-off and compare it with the steady-state remainder of the run.
- **CSP: A ratio compares two quantities: what are the unmet Interests compared with?**
- **Forwarding overhead:** application relay load across the relay nodes, normalised by delivered Data bytes, together with a decomposition into explicit flooding traffic and NLSR control traffic.
- **CSP: What is a relay node? This term has not been defined.**

B. Results

CSP: I think we need to separate discussion of baseline results, using NLSR with various parameter tunings, from the results using OptoFlood. Directly comparing them makes limited sense because the differences are so large.

The results show that baseline NDN exhibits long service interruptions after producer movement. OptoFlood reduces these interruptions to short transients while keeping the additional network load bounded.

1) *Service Continuity:* Under baseline forwarding, each hand-off is followed by a sharp drop in delivered throughput. The consumer continues to issue Interests, but receives little or no Data during the outage interval. Recovery appears only after routing convergence catches up. Figures 8 and 9 show disruption on the order of tens of seconds.

With OptoFlood, the stream remains continuous except for a narrow disturbance around each hand-off. Figure 10 shows recovery on the order of tens of milliseconds. The throughput trace in Figure 11 is correspondingly stable across the mobility events.

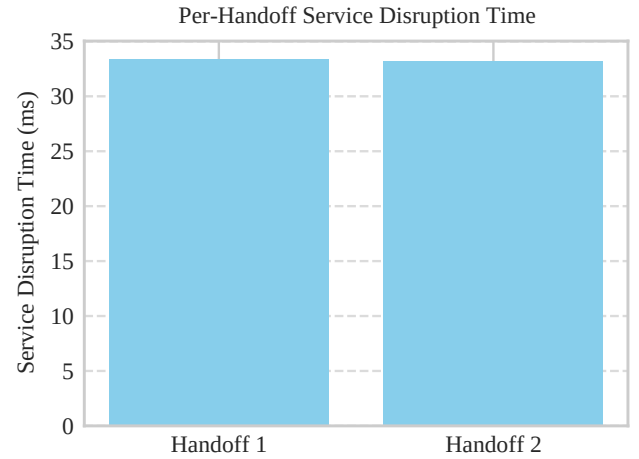


Fig. 10. Per-handoff service disruption with OptoFlood. **CSP: This plot is light on information. Is it possible to plot service disruption time against time taken for the handover to complete, or variation with topology changes, or something?**

2) *Delivery Reliability:* The unmet-interest results follow the same trend. In the baseline case, the immediate post-handoff window contains a high fraction of unsatisfied requests. The consumer continues to issue Interests, but stale forwarding state prevents many of them from being satisfied before their deadline. Figure 12 captures this behaviour directly.

Under OptoFlood, the unmet-interest ratio in the hand-off window falls close to the steady-state level, as shown in Figure 13. This indicates that temporary flooded paths and short-lived forwarding hints are sufficient to sustain delivery while routing converges. The steady-state ratio remains low as well. Normal forwarding therefore remains stable after a valid path has been re-established. **CSP: Again, these plots would be more convincing if they showed a trend as handover**

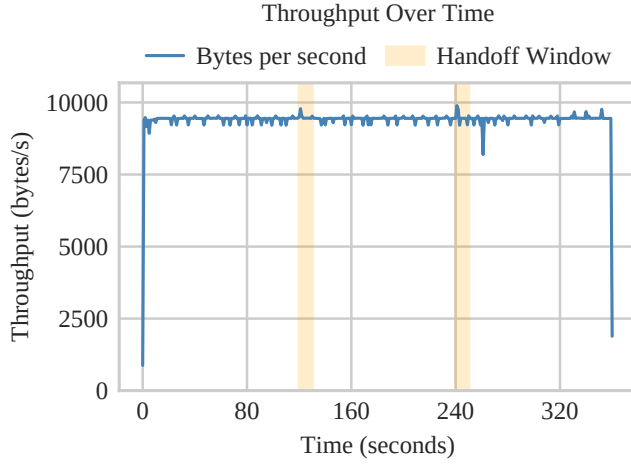


Fig. 11. Throughput continuity with OptoFlood.

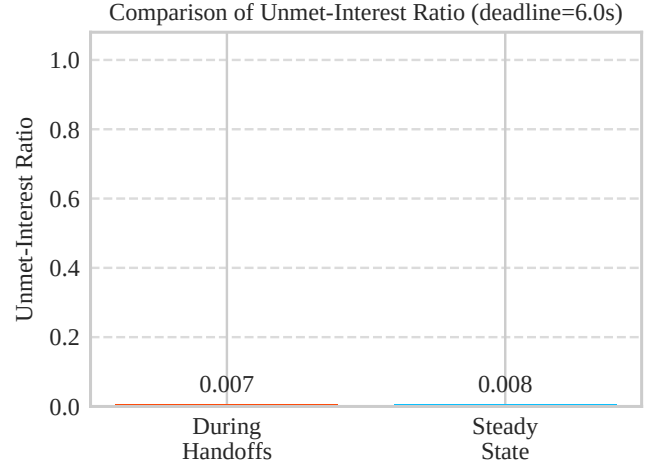


Fig. 13. Unmet-interest ratio with OptoFlood.

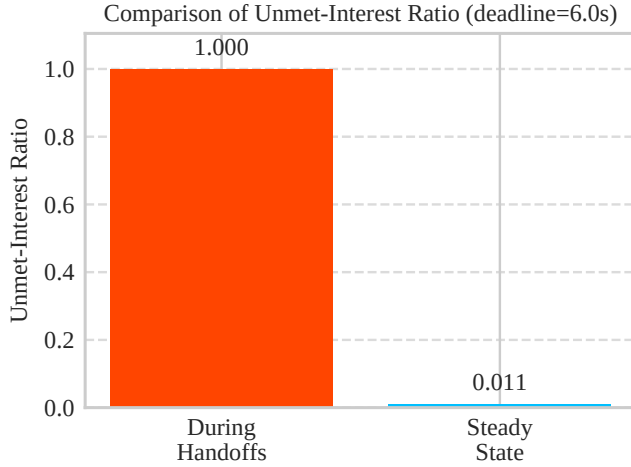


Fig. 12. Unmet-interest ratio under baseline forwarding.

time changed, or as the topology changed, or similar.

3) *Forwarding Overhead and Control Activity*: The overhead figures report network activity of relay load. Figures 14a and 15a show the non-flood and flooded relay traffic for Interests and Data, and also show the NLSR control messages. Figures 14b and 15b summarise the same quantities over the hand-off windows and the full run. The summary panels also report the forwarding-cost ratio and flood share for each window.

Two features are clear in the OptoFlood traces. First, the additional relay load is concentrated around the hand-offs in Figure 15a. The flooded Interest and Data packets rise during recovery and then subside. Second, the full-run summary in Figure 15b remains in the same general range as the baseline. The mechanism therefore adds traffic at the points where recovery occurs, rather than raising relay load throughout the run. The NLSR control messages also appear earlier under OptoFlood, which is consistent with prompt Fast-LSA

triggering after temporary paths are discovered.

The baseline overhead figures show a different pattern. In the immediate post-handoff windows, the consumer may receive no Data. The forwarding-cost ratio is therefore undefined in those windows, as shown in Figure 14b. This reflects the severity of the disruption. Figure 14a still shows routing control activity, but it tends to appear later, after the disruption has already persisted for much longer.

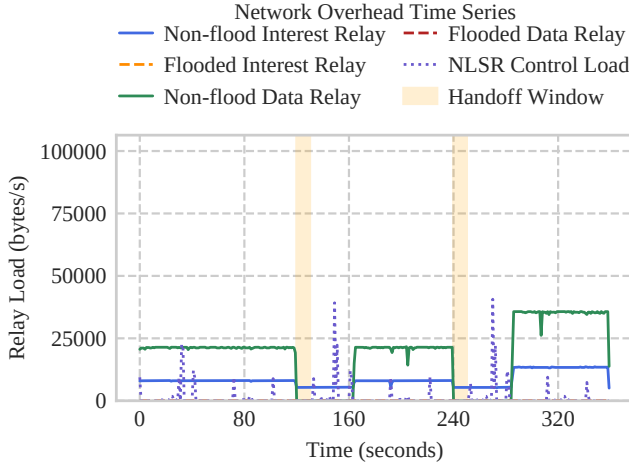
Taken together, the overhead results are consistent with the design target. OptoFlood introduces additional traffic during recovery. That traffic is temporally localised and remains bounded. The figures therefore indicate that the gain in continuity is achieved through short-lived overhead around mobility events, rather than through a persistent increase in relay load.

Overall, the evaluation shows that OptoFlood shortens producer hand-off disruption from long outages to brief transients. It keeps unmet requests close to the steady-state regime during mobility. The additional forwarding and control traffic remains bounded.

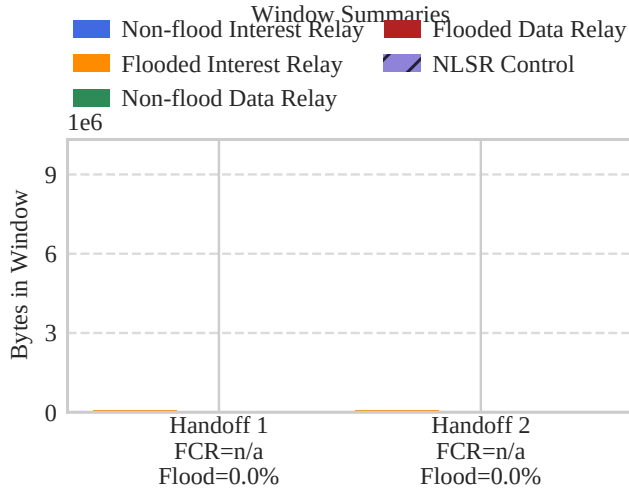
V. RELATED WORK

The management of producer mobility in NDN has attracted significant research attention [3], [4], [6]. Various solutions have emerged, generally categorised according to their underlying mechanisms. Here, we summarise three representative approaches—KITE, AFIRM, and MAP-Me—and highlight their respective strengths and limitations.

KITE [9] is a trace-based mobility support scheme that leverages an immobile rendezvous point and the stateful NDN forwarding plane. A mobile producer maintains soft state by exchanging authenticated Interest/Data messages that establish a hop-by-hop trace toward its latest attachment point, allowing subsequent Interests to follow the trace after reaching the rendezvous point. This approach reduces the reliance on slow global routing convergence and keeps data retrieval largely transparent to applications. However, KITE introduces a dependency on a reachable rendezvous point and periodic

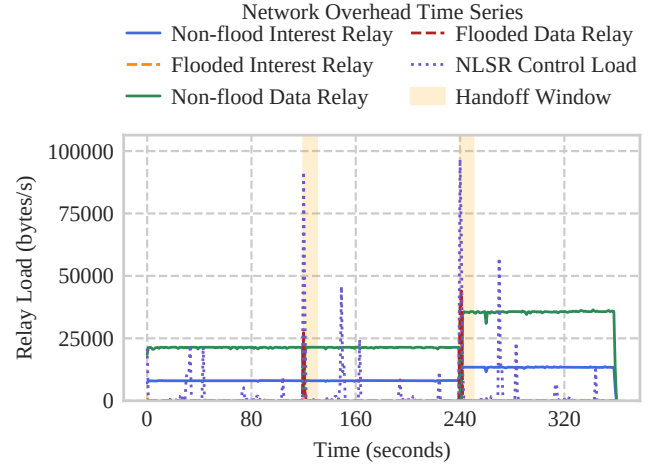


(a) Time-series decomposition by packet type.

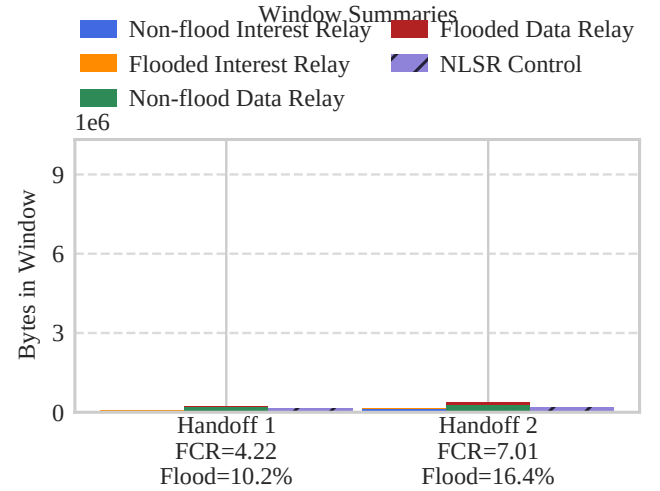


(b) Window summaries by packet type.

Fig. 14. Forwarding and control load under baseline forwarding.



(a) Time-series decomposition by packet type.



(b) Window summaries by packet type.

Fig. 15. Forwarding and control load with OptoFlood.

signalling to refresh soft state, which can become a scalability concern under frequent mobility or large numbers of mobile producers.

AFIRM (Adaptive Forwarding-based Link Recovery for Mobility) [7] addresses producer mobility by repairing forwarding behaviour through adaptive forwarding and local recovery, aiming to re-establish usable paths without introducing a permanent anchor. By reacting to mobility-induced disruptions and adjusting forwarding decisions, AFIRM can reduce service interruption for localised micro-mobility. However, its effectiveness depends on the timely detection and stability of local recovery in highly dynamic conditions; frequent mobility may also introduce additional signalling and forwarding overhead.

MAP-Me (Managing Anchor-less Producer Mobility) [8], [17] provides anchor-less mobility support by handling producer location updates in the data plane. Upon producer

movement, MAP-Me propagates update information hop-by-hop to re-direct affected forwarding paths, avoiding the delays associated with purely routing-plane convergence. MAP-Me is therefore well-suited to latency-sensitive micro-mobility scenarios. However, in frequent or high-intensity mobility, repeated updates can generate noticeable overhead and may require careful tuning to balance responsiveness against network cost.

In terms of route updates, KITE maintains soft-state traces rooted at a rendezvous point, while AFIRM focusses on local forwarding recovery, and MAP-Me proactively updates forwarding state in the data plane upon movement. All three differ fundamentally from baseline NDN deployments where producer relocation requires routing-plane updates (e.g., NLSR prefix re-advertisement) and subsequent global convergence.

MoQ/QUICR (Media over QUIC) [20] proposes a relay-assisted publish/subscribe architecture over QUIC for low-

latency media delivery. Producers publish named media objects, and consumers subscribe; relays can cache and fan out objects. Mobility is handled primarily via endpoint re-attachment (e.g., QUIC connection migration and re-subscription), offering fast application-level continuity. However, this is an overlay design: it does not rely on the PIT/FIF state of NDN in the network, nor does it provide an in the network mechanism to rescue pending requests along disrupted forwarding paths after producer movement. Thus, MoQ/QUICR shares NDN-like ideas (naming and caching), but targets a different layer and problem scope.

In comparison, our OptoFlood solution offers a flood-based mechanism specifically designed for dynamic producer mobility scenarios, employing controlled flooding of *both* Data and Interest packets immediately following producer movement. Unlike KITE, OptoFlood does not depend on a rendezvous/anchor node, avoiding central dependency in the forwarding workflow. Compared to AFIRM and MAP-Me, which focus on forwarding recovery or explicit update propagation, OptoFlood prioritises *immediate* temporary reachability: (i) mobility-marked Data packets are locally flooded to rescue pending Interests on the old path, and (ii) forwarders trigger scoped Interest flooding when forwarding state is missing, to rapidly discover the producer’s new attachment point. Once a working face is confirmed, forwarders install short-lived TFIB state to quickly cut back to unicast. Finally, OptoFlood couples these temporary paths with the routing plane by triggering a short-lived, locally scoped Fast-LSA update via NLSR, accelerating convergence while allowing normal NLSR advertisements to eventually take over.

VI. CONCLUSIONS

This paper presented *OptoFlood*, a controllable flooding mechanism for producer mobility in Named Data Networking. OptoFlood addresses the key weakness of routing-based mobility handling: after a producer handover, forwarding state and routing information can remain stale for seconds, during which Interests may be discarded and Data may fail to return along invalidated reverse paths. Rather than waiting for global convergence, OptoFlood provides immediate localised recovery by establishing temporary bidirectional reachability and then accelerating routing convergence through lightweight integration with NLSR.

OptoFlood combines two complementary mechanisms. First, mobility-marked *Data packet flooding* disseminates Data generated around a handover with bounded scope and duplicate suppression, improving the likelihood that pending Interests issued before the move are satisfied without relying on global route updates. Second, *Interest packet flooding* is triggered by forwarders only as a last resort, when both FIB and TFIB provide no usable next hop, and is confined by native hop limits and per-node de-duplication. Intermediate forwarders opportunistically install short-lived TFIB hints to quickly revert subsequent Interests back to unicast forwarding once a viable face is observed. Finally, OptoFlood couples these temporary paths to the control plane via a Fast-LSA

trigger, installing short-lived routing entries through NLSR to shorten the overall convergence period while allowing standard NLSR advertisements to eventually take over.

Overall, OptoFlood aims to keep mobility recovery *local, short-lived, and explicitly bounded*: flooding is scoped (hop-limited), suppressed (duplicate filtering), and constrained (rate limiting), while cryptographic integrity is preserved by keeping immutable identifiers inside signed Data MetaInfo extensions and leaving hop-by-hop controls in the LP header for in-network processing. These design choices make OptoFlood practical for latency-sensitive micro-mobility settings, where rapid continuity matters and mobility events occur frequently enough that waiting for global convergence alone is insufficient.

A. Long Term Impact Assessment

1) Continuous optimisation of network performance:

- **Efficiency Improvements:** Because OptoFlood restores temporary reachability quickly after handover, it can reduce the duration of “blind” retransmission phases where consumers repeatedly re-express Interests while forwarding state is stale. Over time, this can translate into smoother application performance and fewer bursts of recovery traffic during mobility.
- **Enhanced Reliability:** By providing a mechanism to rescue pending Interests and by enabling forwarders to cache short-lived forwarding hints (TFIB), the network becomes more resilient to transient disruptions caused by producer relocation, especially when mobility remains local (within a few hops) and frequent.

2) Resource Usage and Network Stability:

- **Resource Optimization:** OptoFlood is based on explicitly bounded flooding with a short lifetime. TFIB entries and Fast-LSA updates are also short-lived, limiting long-term state growth and reducing the risk of persistent routing pollution. This provides a pragmatic balance between rapid recovery and overhead.
- **Stability Analysis:** Although flooding is controlled, its interaction with retransmission behaviour, caching, and routing updates can influence stability under high load or dense mobility. The throttling and de-duplication mechanisms in both the forwarding plane (FloodId/Nonce caches, rate limits) and the routing plane (Fast-LSA throttling) are therefore essential to prevent oscillation and excessive control traffic, and should be considered carefully in larger deployments.

ACKNOWLEDGEMENTS

This work was supported in part by Rakuten Mobile.

REFERENCES

- [1] V. Jacobson, D. K. Smetters, J. D. Thornton, M. F. Plass, N. H. Briggs, and R. L. Braynard, “Networking named content,” in *Proceedings of the 5th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, 2009, pp. 1–12.
- [2] L. Zhang, D. Estrin, J. Burke, V. Jacobson, J. Thornton, G. Tsudik, and B. Zhang, “Named data networking,” *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 66–73, Jul. 2014.

- [3] A. Abrar, A. S. C. M. Arif, and K. M. Zaini, "A systematic analysis and review on producer mobility management in named data networks: Research background and challenges," *Alexandria Engineering Journal*, vol. 69, pp. 785–808, 2023.
- [4] B. Feng, H. Zhou, and Q. Xu, "Mobility support in named data networking: A survey," *EURASIP Journal on Wireless Communications and Networking*, vol. 2016, no. 220, p. 220, Sep. 2016.
- [5] G. Tyson, N. Sastry, I. Rımac, R. Cuevas, and A. Mauthe, "A survey of mobility in information-centric networks: Challenges and research directions," in *Proceedings of the 1st ACM Workshop on Emerging Name-Oriented Mobile Networking Design (NoM)*, Jun. 2012, pp. 1–6.
- [6] Y. Zhang, A. Afanasyev, J. Burke, and L. Zhang, "A survey of mobility support in named data networking," in *Proc. IEEE INFOCOM Workshops*, Apr. 2016, pp. 83–88.
- [7] M. Meddeb, A. Dhraief, A. Belghith, T. Monteil, K. Drira, and S. Gannouni, "AFIRM: Adaptive forwarding based link recovery for mobility support in NDN/IoT networks," *Future Generation Computer Systems*, pp. 351–363, May 2018.
- [8] J. Augé, G. Carofiglio, G. Grassi, L. Muscariello, G. Pau, and X. Zeng, "MAP-Me: Managing anchor-less producer mobility in information-centric networks," arXiv:1611.06785, Nov. 2016.
- [9] Y. Zhang, Z. Xia, S. Mastorakis, and L. Zhang, "KITE: producer mobility support in named data networking," in *Proceedings of the Conference on Information Centric Networking (ICN)*. Boston, MA, USA: ACM, Sep. 2018, pp. 125–136.
- [10] D. Trossen and G. Parisi, "Designing and realizing an information-centric internet," *IEEE Communications Magazine*, vol. 50, no. 7, pp. 60–67, Jul. 2012.
- [11] G. Xylomenos, C. N. Ververidis, V. A. Siris, N. Fotiou, C. Tsilopoulos, X. Vasilakos, K. V. Katsaros, and G. C. Polyzos, "A survey of information-centric networking research," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 2, pp. 1024–1049, 2014.
- [12] C. Yi, A. Afanasyev, I. Moiseenko, L. Zhang, and B. Zhang, "A case for stateful forwarding plane in named data networking," *Computer Communications*, vol. 36, no. 7, pp. 779–791, 2013.
- [13] A. K. M. M. Hoque, S. O. Amin, A. Alyyan, B. Zhang, L. Zhang, and L. Wang, "NLSR: Named-data link state routing protocol," in *Proceedings of the Conference on Information-Centric Networking (ICN)*, Aug. 2013.
- [14] Z. Zhu and A. Afanasyev, "Let's chronosync: Decentralized dataset state synchronization in named data networking," in *Proceedings of the 21st IEEE International Conference on Network Protocols (ICNP)*, Oct. 2013.
- [15] D. Johnson, C. Perkins, and J. Arkko, "Mobility support in IPv6," IETF RFC 3775, Jun. 2004.
- [16] S. Gundavelli, K. Leung, V. Devarapalli, K. Chowdhury, and B. Patil, "Proxy mobile IPv6," IETF RFC 5213, Aug. 2008.
- [17] J. Augé, G. Carofiglio, G. Grassi, L. Muscariello, G. Pau, and X. Zeng, "MAP-Me: Managing anchor-less producer mobility in information-centric networks," *IEEE Transactions on Network and Service Management*, vol. 15, no. 2, pp. 596–610, Jun. 2018.
- [18] A. Afanasyev, P. Mahadevan, I. Moiseenko, E. Uzun, and L. Zhang, "Interest flooding attack and countermeasures in named data networking," in *IFIP Networking Conference*, May 2013.
- [19] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM Workshop on Hot Topics in Networks (HotNets)*, Oct. 2010.
- [20] C. Jennings and S. Nandakumar, "QuicR: Media delivery protocol over QUIC — architecture," IETF Internet-Draft (draft-jennings-moq-quic-arch-01), Jul. 2022.